
Testing

— Entornos de desarrollo —

Índice

Introducción

Tipos

Herramientas

Ejemplos

Introducción

Testing

El **testing** en programación es una fase fundamental del desarrollo de software que ayuda a garantizar que el código funcione correctamente. Su objetivo es **detectar errores** antes de que el software llegue a producción, mejorando la calidad y fiabilidad del sistema.

Tipos

Pruebas

Después de la implementación, se realizan pruebas exhaustivas para detectar y corregir errores (bugs) y garantizar que el software funcione correctamente. Esto incluye pruebas de unidad, pruebas de integración, de sistema y pruebas de aceptación:

- Pruebas unitarias
- Pruebas de integración
- Pruebas de sistema
- Pruebas de aceptación

Pruebas. Pruebas unitarias

Las pruebas unitarias **son las más básicas y se enfocan en verificar que las funciones o métodos individuales** dentro de un módulo específico del código funcionen de manera correcta. Estas pruebas se realizan de forma aislada, sin depender de otros componentes del sistema.

1. `calcularCostoTotal(estancia, cargoExtra)`:
 - `calcularCostoTotal(3, 50)`. Si el precio por noche es \$100, el resultado esperado es \$350 (3 noches a \$100 + \$50 de cargo extra).
 - `calcularCostoTotal(5, 0)`. Si el precio por noche es \$100, el resultado esperado es \$500 (5 noches a \$100, sin cargo extra).

Pruebas. Pruebas de integración

Las pruebas de integración examinan **cómo funcionan en conjunto diferentes módulos o componentes que interactúan entre sí**. Estas pruebas se centran en identificar errores en la comunicación entre módulos.

Pruebas. Pruebas de sistema

Las pruebas de sistema evalúan **el sistema completo en su totalidad**, verificando que todos los módulos y componentes funcionen juntos de acuerdo con los **requisitos funcionales**. Estas pruebas suelen realizarse en un **entorno que simula el ambiente de producción**.

Pruebas. Pruebas de Aceptación del Usuario (UAT)

Las pruebas de aceptación del usuario (UAT) **son realizadas por el cliente o un grupo de usuarios finales**. Estas pruebas se enfocan en verificar que el software cumple con los requisitos y expectativas del cliente, y en evaluar si es intuitivo y fácil de usar.

Herramientas

Python nos ofrece varias

Python ofrece varias herramientas para realizar pruebas:

- **unittest:** Módulo incorporado en Python para pruebas unitarias.
- **pytest:** Framework flexible y potente para testing.
- **doctest:** Permite escribir pruebas dentro de docstrings.

pytest

```
import pytest

def suma(a, b):
    return a + b

def test_suma():
    assert suma(2, 3) == 5
```

Cobertura de código

La cobertura de código es un parámetro con el que podrás saber qué parte de tu fuente se ha sometido a pruebas.

Cobertura de código

Cobertura de **funciones**: la cantidad de funciones definidas que se abarcan.

Cobertura de **instrucciones**: cuántas instrucciones del programa se han ejecutado.

Cobertura de **ramas**: cuántas ramas de las estructuras de control (instrucciones IF, por ejemplo) se han ejecutado.

Cobertura de **condiciones**: en cuántas subexpresiones booleanas se ha comprobado el valor verdadero o falso.

Cobertura de **líneas**: cuántas líneas de código fuente se han probado.

Cobertura de código

Stmts: El número total de líneas de código en el archivo.

Miss: El número de líneas no cubiertas por las pruebas.

Cover: El porcentaje de líneas cubiertas por las pruebas.

Missing: Las líneas específicas que no fueron cubiertas.

Comprobaciones típicas

`.assertEqual(a, b)`: Verifica la igualdad de ambos valores.

`.assertTrue(x)`: Verifica que el valor es True.

`.assertFalse(x)`: Verifica que el valor es False.

`.assertIs(a, b)`: Verifica que ambas variables son la misma (ver operador `is`).

`.assertIsNone(x)`: Verifica que el valor es None.

`.assertIn(a, b)`: Verifica que `a` pertenece al iterable `b` (ver operador `in`).

`.assertIsInstance(a, b)`: Verifica que `a` es una instancia de `b`.

`.assertRaises(x)`: Verifica que se lanza una excepción.

unittest

```
import unittest

def suma(a, b):
    return a + b

class TestSuma(unittest.TestCase):
    def test_suma(self):
        self.assertEqual(suma(2, 3), 5)

if __name__ == '__main__':
    unittest.main()
```

Vamos a probarlo

TestSuma -> coverage run -m unittest .\SumarTest.py

ToDo List